

Final Report

Project 2522: Digital Twin for Cooperative Robotic Additive Manufacturing

ECE Team: Tyler Nardi (Electrical and Computer Engineering), Kalin Kochnev (Robotics Engineering)

Sponsor: School of Mechanical, Aerospace, and Manufacturing Engineering
(farhad.imani@uconn.edu)

Date: May 6th, 2025

Abstract

Additive manufacturing and robotic systems have both become increasingly popular in industry for manufacturing complex geometries. While traditional AM systems can manufacture more complex geometries than subtractive manufacturing, there is still room for improvement through the utilization of robotic arms in AM. Particularly, the additional degrees of freedom afforded by robotic arms allow for multiplanar and nonplanar slicing methods, which can improve the geometric accuracy and strength of manufactured parts. Robotic arms also allow for a larger build volume, particularly when combined into a collaborative robotic arm system. Utilizing multiple robotic arms will allow for increased manufacturing speed for large parts, as well as increasing the potential build volume. Additional motion planning considerations need to be taken when planning with multiple arms to ensure they do not collide. The slicing and motion planning tools will be developed along with a digital twin that will monitor the manufacturing process to increase the geometric accuracy, strength, and print speed of manufactured parts.

Table of Contents

Abstract	2
List of Figures/Tables	4
Glossary	6
Introduction	7
Introduction to Slicing.....	8
Introduction to Motion Planning	10
Problem Statement	12
Statement of Need.....	12
Requirements/Specifications.....	13
Approach and Design	16
Manufacturing Pipeline	Error! Bookmark not defined.
Slicing Approach.....	16
Motion Planning Approach	19
Project Management	24
Tasks and Subtasks.....	24
RACI Chart	25
Timeline.....	26
Results	28
Slicing Results.....	28
Motion Planning Results	31
Summary	34
Appendix	35
Senior Design Checklist	35
Bibliography	37

List of Figures/Tables

Figure 1: Toolpath generated for a 3D model using a planar slicer (PrusaSlicer) with 100% infill	9
Figure 2: Multiplanar slicing of a rectangle in the X-Y and Y-Z planes.	9
Figure 3: Multi-arm C-RAM configurations for a) low overlap (e.g., large-scale fabrication with homogeneous tooling) and b) high overlap (e.g., multi-material fabrication with heterogeneous tooling) [1].....	11
Figure 4: Decomposition of software tools to be developed for collaborative robotic additive manufacturing.....	13
Figure 5: The complete pipeline for manufacturing a part, starting with part decomposition, then slicing, and robot coordination	16
Figure 6: A 3D model of a cone that is segmented into three segments. As the robots manufacture, they converge towards the center into a deadlock. As the layer height increases, the layer area decreases, and conflict increases.	20
Figure 7: Coordination space representation of robot tasks. Robot 1 tasks are the columns and Robot 2 tasks are the rows. Black squares are collision between task pairs.	22
Figure 8: The optimal solution in a coordination space with no conflicts are the two main diagonals.	23
Figure 9: Gaps between diagonals represent a transition between tasks in coordination space.	23
Figure 10: Coordination space representation converted into a bipartite graph matching problem to be solved using maximum flow.	24
Figure 11: Gantt Chart for ECE Team 2522	27
Figure 12: Rectangular Prism sliced using a lambda function representing a triangular waveform. The triangular waveform is propagated in the x-axis and y-axis. The lambda function is not differentiable, but the inversion process is successful in restoring the original model.....	28
Figure 13: Slicing Process Shown for an arch being sliced with a quadratic function. (a) shows the original arch, and (b) shows the arch after being transformed before being sent to the backend. Finally, (c) shows the final g-code generated after the inversion is complete.	29
Figure 14: Nonplanar model manufactured with non-uniform surface.....	29
Figure 15: Nonplanar manufacturing of a rectangular prism using a quadratic surface that required rotating the end effector.	30
Figure 16: Extrusion of nonplanar model with the hot end vertical since the slicing surface used does not cause interference.	30
Figure 17: Manufacturing a nonplanar model with the extruder oriented normal to the motion and away from the existing model.	31
Figure 18: In red: robot, In black: path extruded by robot on current layer, In gray: previous layers extruded by robots.	32

Figure 19: Upper and lower diagonal solutions for two different types of randomly generated coordination spaces. First two figure columns are randomly generated rectangular conflicts while next two figure columns are noisy coordination spaces. 33

Table 1: Specifications for the Digital Twin for CRAM 14

Table 2: RACI Chart for ECE Team 2522 26

Glossary

AM: Additive Manufacturing .

CRAM: Collaborative Robotic Additive Manufacturing.

Digital Twin: A virtual model of a physical system that uses real-time data to simulate performance and modify process parameters.

FFF: Fused Filament Fabrication.

G-code: Widely used programming language for controlling traditional computer numerical control manufacturing machines and 3D printers.

Infill: The internal structure that fills the volume of a 3D printed component.

Mesh: A digital model of a 3D object consisting of vertices, edges, and faces to define an object's shape.

Multiplanar Slicer: A 3D printing technique that involves slicing a model using multiple planes facing different directions.

Nonplanar Slicer: A 3D printing technique that involves generating curved layers.

RAM: Robotic Additive Manufacturing.

ROS: Robot Operating System.

RTSP: Robot Task Sequencing Problem.

Slicer: A program that converts a 3D model into a set of commands that can be used by a 3D printer.

STL: Stereolithography. A file format that describes a raw, unstructured triangulated surface

TSP: Traveling Salesman Problem.

Introduction

Additive manufacturing (AM) has become increasingly common in the past few decades. Additive manufacturing functions by depositing material using thin cross sections stacked on top of one another. Legacy manufacturing techniques are often subtractive or formative. Subtractive methods whittle away material from a piece of stock while formative methods use a mold which casts material into its shape. Additive manufacturing has many benefits that make it attractive in certain applications like aerospace or the medical field. AM is inexpensive and can be a fully automated process which is ideal as a prototyping tool. However, AM isn't limited to prototyping only. It is capable of generating geometries that would be impossible using any other technique. For instance, internal structures allow for the creation of lightweight components without sacrificing strength by replacing a solid interior with a lattice interior, cutting back on material costs. Additionally, a wide variety of different AM techniques exist which can allow for printing with vastly different materials, anything between thermoplastics, metals, or in the future, even organs.

With all the benefits listed, there is much room for improvement. Current AM technologies enable new geometries to be manufactured but there still exists several kinds of geometries that are problematic. For instance, overhanging structures require support materials that require post-processing to remove that material. Additionally, depending on the technique used to manufacture a part, additively manufactured parts are stronger along certain orientations than others, which adds a host of additional complications and decreases AM's viability as a manufacturing method.

A new area of research, known as Robotic Additive Manufacturing (RAM), makes use of robotic manipulators instead of gantry systems. The introduction of manipulators for manufacturing has the potential to address many of the shortcomings of standard AM.

Our senior design team is tasked with improving Dr. Farhad Imani's RAM platform. The electrical engineering team is tasked with implementing new techniques for non-planar and multi-planar slicing, compared to standard planar slicers existing in AM systems today. Additionally, the electrical team will implement motion planning algorithms to coordinate multiple manipulators capable of manufacturing in order to parallelize the workload and increase throughput. We are also in collaboration with a senior design team from the Department of Computer Science and Engineering who will implement 3D model decomposition and visualization tools to streamline RAM workflows.

In this report we familiarize the reader with common concepts and challenges in additive manufacturing. We then go into the details of implementing a cohesive RAM platform and the final deliverables at the conclusion of this senior design project.

Introduction to Slicing

Slicing is the process of using a 3D model to generate a toolpath that can be used with a 3D printer to produce the part. Along with the part, there are a series of configuration settings that can be used to determine the particular toolpath. These settings include managing the amount of infill generated, the pattern of the infill, as well as the size of each layer and amount of material to extrude. These settings rely on the strength required for the part as well as the physical limitations of the machine being used. The size of layers, speed of the movements and ratio of material to be extruded per unit distance are all dependent on the physical 3D printer. Settings related to the infill generated are more reliant on the requirements for part strength, with more infill leading to a sturdier part.

Traditional 3D printing uses planar slicing to generate these toolpaths. There are already many open-source and commercial options available to complete this task, including PrusaSlicer and Cura Slicer. To develop a toolpath, planar slicing first creates a 2D cross-sections of the model parallel to the X-Y plane. These cross-sections are made at a predetermined offset based on the layer height. After these cross-sections are made, the software algorithmically determines a path to properly cover the required area in the cross-section. Finally, this path is converted into G-code instructions with extrusion amounts attached to the instructions. These instructions are in the same format as those used for subtractive machining, and they are directly interpretable by the 3D printer. An example of a generated toolpath can be seen in the figure below.

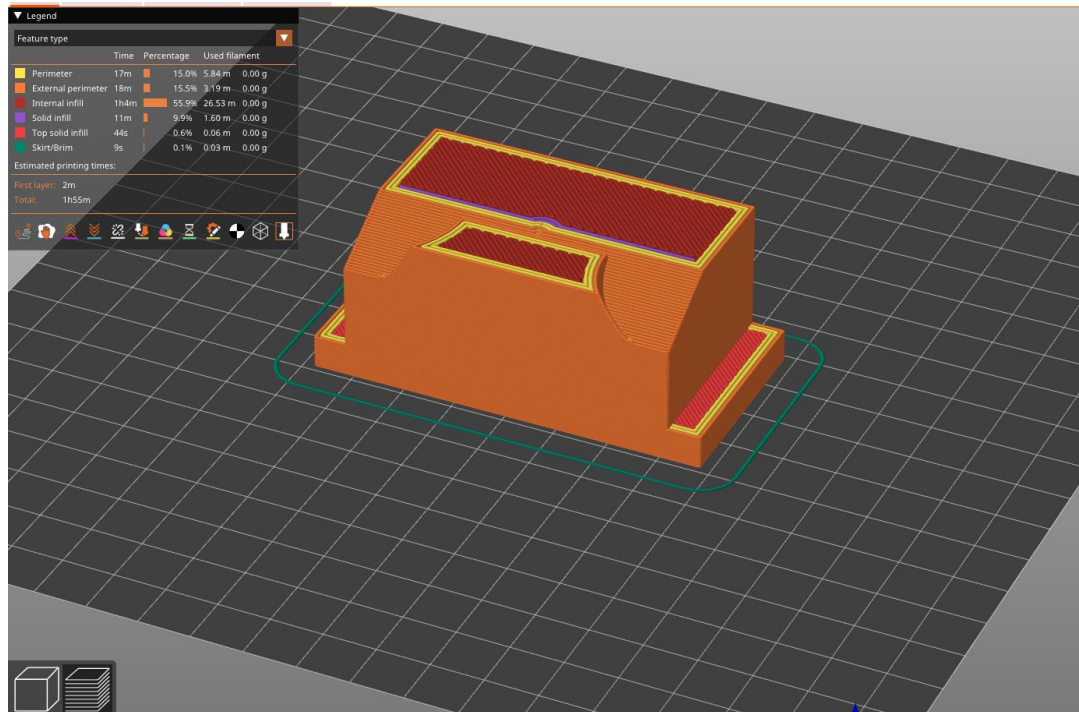


Figure 1: Toolpath generated for a 3D model using a planar slicer (PrusaSlicer) with 100% infill

Beyond planar slicing, there are also multiplanar and nonplanar slicing techniques. Multiplanar slicing is fundamentally like planar slicing, but the key difference is that cross-sections are taken in planes other than the X-Y plane. The selection of different planes can be performed in a continuous or discrete fashion. A finite number of planes could be chosen based on the model geometry. If there is not a clean division in the model to select a few planes, then a curve through the model could be generated with each cross-section being taken such that the normal vector of the plane is tangential to this curve. An example of discrete multiplanar slicing can be seen in the figure below.

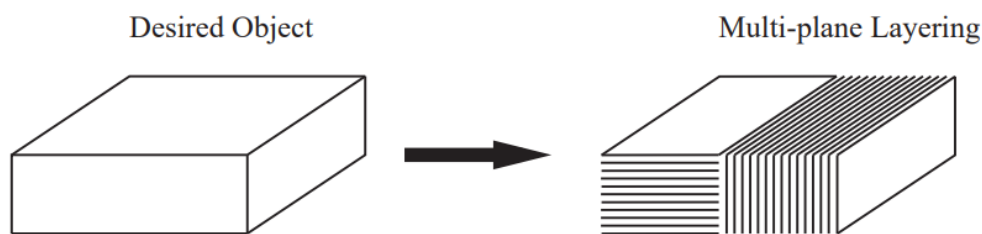


Figure 2: Multiplanar slicing of a rectangle in the X-Y and Y-Z planes.

For nonplanar slicing, rather than using a straight cross-section of the model, a curve that intersects the model is used to generate each layer. This curve can be conformally or arbitrarily defined. For a conformal curve, the top surface of the model is used as a reference for slicing. Each layer follows the form of the top surface, and this method produces parts with

improved surface finish and mechanical strength. For arbitrarily defined nonplanar slicing, a particular surface is not used as a reference. Rather, a function is defined such that a layer is generated along the path of the function. These functions can be chosen by the user on a case-by-case basis, or through the use of heuristics, such as optimizing for mechanical strength, surface finish, or support reduction.

Nonplanar and multiplanar slicing both serve similar purposes of increasing the mechanical strength of the part, increasing the surface quality, or decreasing the amount of support material required. The weakest element of a 3D printed part is layer-wise separation, so orienting the layers parallel to the expected strain to be faced by the fabricated part will result in a relatively stronger part. Aliasing is a common problem in 3D printed parts that have a curve or slope as the Z-height of the part increases. This is because each layer has a finite size, so the curve cannot be made smooth when stepping from one layer to the next. By orienting the layers with this curve, the surface quality can be improved. Lastly, support material is required when printing large overhangs as the filament will be deposited over air and fall without support material. To remedy this, the overhangs could be printed off of the existing part at an angle.

Introduction to Motion Planning

Motion planning is the calculation of a robot trajectory to meet specific objectives. In the simplest case, the objective may be to determine the sequence of movements to go from point A to point B without collisions.

Standard slicers have a planner that calculates the movements of the tool head (an extruder) to cover cross-sectional regions of a part. This is referred to as the tool path plan. The tool path plan output by slicers cannot be used by our system directly because it does not consider collisions between multiple agents as well as the inverse kinematics of each arm.

There are multiple sources for potential collisions. A robot arm may collide with itself since the links have thickness. The arm may also collide with the part that it is manufacturing since the part geometry changes as more material is deposited. And finally, the most important kind, collisions between multiple robot arms are possible.

The closer two robot arms are, the more likely a collision is to occur. This can be visualized as a region of conflict between two arms that grows the closer the arms are. The areas of conflict also change depending on the size of the part being manufactured. A very small part will have very high conflict. Overlap of the workspace cannot be avoided when

manufacturing a part because an interface between 3D prints is required to maintain strength.

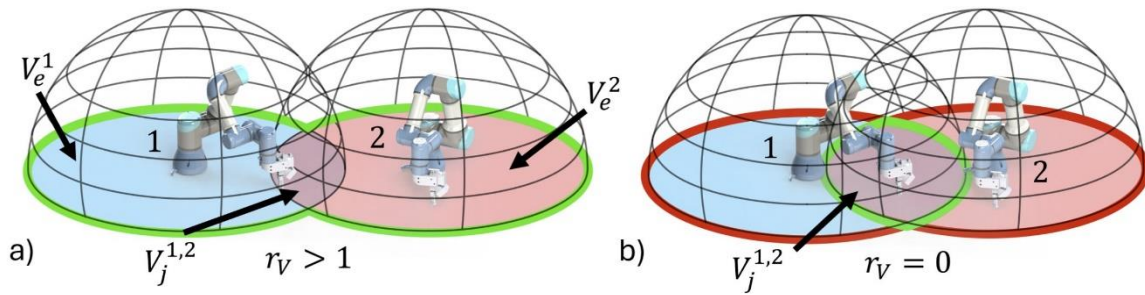


Figure 3: Multi-arm C-RAM configurations for a) low overlap (e.g., large-scale fabrication with homogeneous tooling) and b) high overlap (e.g., multi-material fabrication with heterogeneous tooling) [1]

While our problem has many commonalities with the problem of multi-agent motion planning, our system is in a well-controlled environment with few unknowns. Every agent has perfect knowledge of the other's state and planned maneuvers, so pre-planning of the trajectories is possible. Unanticipated external disturbances are minimal since the system is used for manufacturing purposes. And much more information is available to guide the motion planner than simple A to B goal points since the slicer has done the work ahead of time.

It is important to note that there are several important differences between our motion planning problem and standard point A to point B motion planning problems. Standard techniques typically seek to find any valid, non-colliding path between a start and end point. The motion planner would select the path to follow out of the infinitely many options. This is in contrast to our problem where the toolpaths are already determined by a slicer. There is no need to search for a path. The motion planning question becomes how to coordinate these arms to execute an already known/fixed path while minimizing manufacturing downtime for each arm. This is fundamentally a different problem compared to conventional motion planning.

Coordination of the motion of multiple robot arms without collisions is still an open research problem. Currently there are three approaches in how to formulate this problem: the Robot Task Sequencing Problem (RTSP), Path Coordination, and Space Partitioning. The RTSP is a variation of the traveling salesman problem. Path coordination formulates the problem as a standard motion planning problem except in what is known as a "coordination space" to determine how to delay the motion along a path to avoid collisions. Space partitioning methods typically split the workspaces of each arm into safe and unsafe regions that then have scheduled access to avoid collisions. The benefits and drawbacks of each method are discussed later in the report.

Additionally, performant collision checking algorithms are vital in having a performant motion planner. Approximations are often used to simplify the geometries involved, such as

using bounding spheres or capsules. Collision detection routines are available in ROS through their package system and can be modified to suit our needs.

Problem Statement

Statement of Need

Additive manufacturing is a widespread manufacturing technique that can produce geometries that are not possible with conventional manufacturing. Most systems today make use of gantry systems with an attached extruder. While many geometries are achievable, there are still many downsides to using gantry-based machines. Parts with large overhangs require support material to avoid material deposition in mid-air. These overhangs can be troublesome to remove depending on the type of material. The stronger the material, the more post-processing is required. To avoid supports, the manufacturer may be able to re-orient the component along a different axis so that the angle of overhangs is minimized. This method has its own issues. Components manufactured by gantry systems are anisotropic, meaning they are stronger along one axis over another. It is often a requirement to manufacture a 3D printed component along a certain axis to maintain structural integrity. Not to mention that the resolution of the layer heights can affect the dimensions of a part if it is oriented along a different axis.

Another technique used in this situation is part decomposition. A 3D model can be decomposed into several smaller segments that are manufactured individually and then joined together in an assembly. This can remove the need for support and maintain structural integrity along the proper axes. However, the technique for assembly used can introduce large failure points that would not have been present had it been one contiguous part. Deliberate engineering effort must be spent to ensure that strength is maintained. This defeats the purpose of trying to manufacture a single part on a 3D printer.

Other significant drawbacks of standard gantry systems are fixed build volumes, single tool-heads, and single materials. There are many use cases that might want mobile and unconstrained 3D printing like construction. Multiple material printing can be used to augment strength. And multiple tool-heads can be used to multiply the speed of a 3D printing system.

Robotic additive manufacturing (RAM) has the potential to address many of the shortcomings of gantry systems. Since robot arms are not constrained to X-Y motions, the “slices” of a part that are computed in software can be non-planar or multi-planar. This would also theoretically remove the need to physically decompose parts onto separate prints. A part could be digitally and optimally decomposed and printed along different orientations on a single workpiece with no intervention from a designer.

Multiple RAM systems could be combined to work together to manufacture a part at increased speed. This is done by distributing the workload across multiple agents. Or each arm could be responsible for different tasks in the 3D print process such as sensing/feedback or extruding different materials. Robot arms with external sensing can be used to develop sophisticated control algorithms to apply mid-layer and inter-layer feedback. And finally, a mobile robot arm could be used to print an infinitely large volume. However, this is hard to realize in practice.

In the same way that 3D printing revolutionized manufacturing, robotic additive manufacturing has the potential to be the next leap in additive manufacturing. RAM systems will unlock new production capabilities and enable intricate designs and geometries to operate in new and challenging scenarios.

Requirements/Specifications

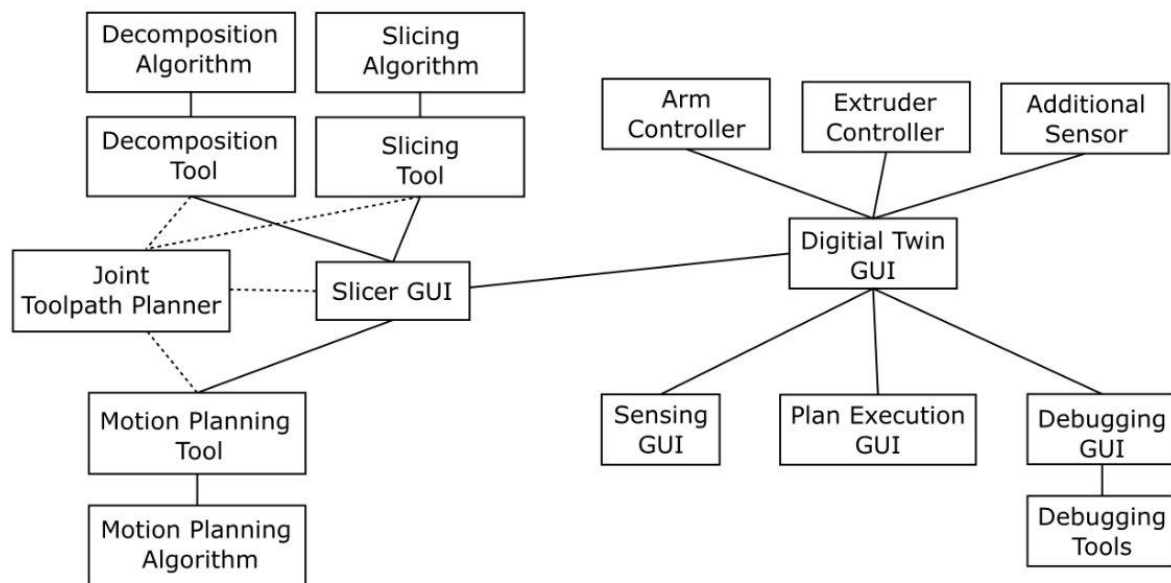


Figure 4: Decomposition of software tools to be developed for collaborative robotic additive manufacturing

The requirements for the project can be broken up into four aspects: Part decomposition, Slicing, Motion Planning, Feedback, and Visualization/Debugging.

The first stage of part decomposition should decompose a single model into multiple segments so that the manufacturing workload can be distributed across multiple robot arms.

The parts could be decomposed according to multiple criteria such as ensuring equal layer completion times or a division that would prevent collisions.

The second stage of part decomposition is closely intertwined with slicing. The purpose of the second stage is to break down each segment even further into regions that would benefit from varying print settings, such as planar, multi-planar, or non-planar slicing.

The slicing task is to develop a framework to enable user-defined nonplanar and multiplanar slicing. The user should be able to arbitrarily define the methods of slicing as planar, multiplanar, nonplanar, or some combination of all three. Additionally, the slicing framework should offer the ability to optimize the slicing methodology based on the model and some heuristics including mechanical strength, surface quality, and support material reduction. Lastly, it should support features used in commonly available slicing solutions, such as infill and layer size configuration.

The motion planning task is to integrate multiple tool path plans generated by the slicer to prevent collisions. The planner will determine the optimal ordering of G-code commands and joint trajectories to prevent collisions, minimize pauses and material retractions, and maintain smoothness while tracing the specified toolpath output by the slicer. Finally, the system will output a series of joint positions and velocities to be executed by the Robot Operating System (ROS) framework MoveIt.

A digital twin representation of the physical system should be created. This digital twin should consume data from the robot arms and external sensors to create a live simulation of the printing process. The digital twin should represent the robot arm, fused filament fabrication extruder head control, material deposition, toolpath planning, motion planning, and sensor feedback. The digital twin should also have data from the slicing and parsing of G-code informing its behavior. This software should be designed in a modular fashion, so multiple different robots, additive manufacturing heads, and sensors can be incorporated.

In the future, there will be a machine learning model created to correlate thermal images with underfill defects and other geometric deviations. The feedback from this model should be useful for modifying the toolpath plan or parameters. It should also be able to inform the boundary surface between separate slicing chunks. This informed toolpath generation and plan modification should result in a quantifiable improvement to the mechanical properties, such as tensile strength and surface roughness.

Table 1: Specifications for the Digital Twin for CRAM

Specification	Value
---------------	-------

Number of robot arms	1-2
Operating system	Linux/Robot Operating System
Motion planning methods and software	Iterative, ROS MoveIt
Planning execution time	1-10 milliseconds
Number of extruders	1-2
Sensing Feedback methods	Temperature, Laser Scanning
Material extrusion	PLA
Hot end temperature	0-300 Celsius
Slicing software	Prusa Slicer
Tensile Strength Relative to Traditional Additive	$\geq 100\%$
Surface Roughness	$\leq$ The same part on a RAM system
Manufacturing Time	$\leq$ The same part on a gantry printer
Geometric Deviations	$\leq$ Geometric Deviations of Baseline

Approach and Design

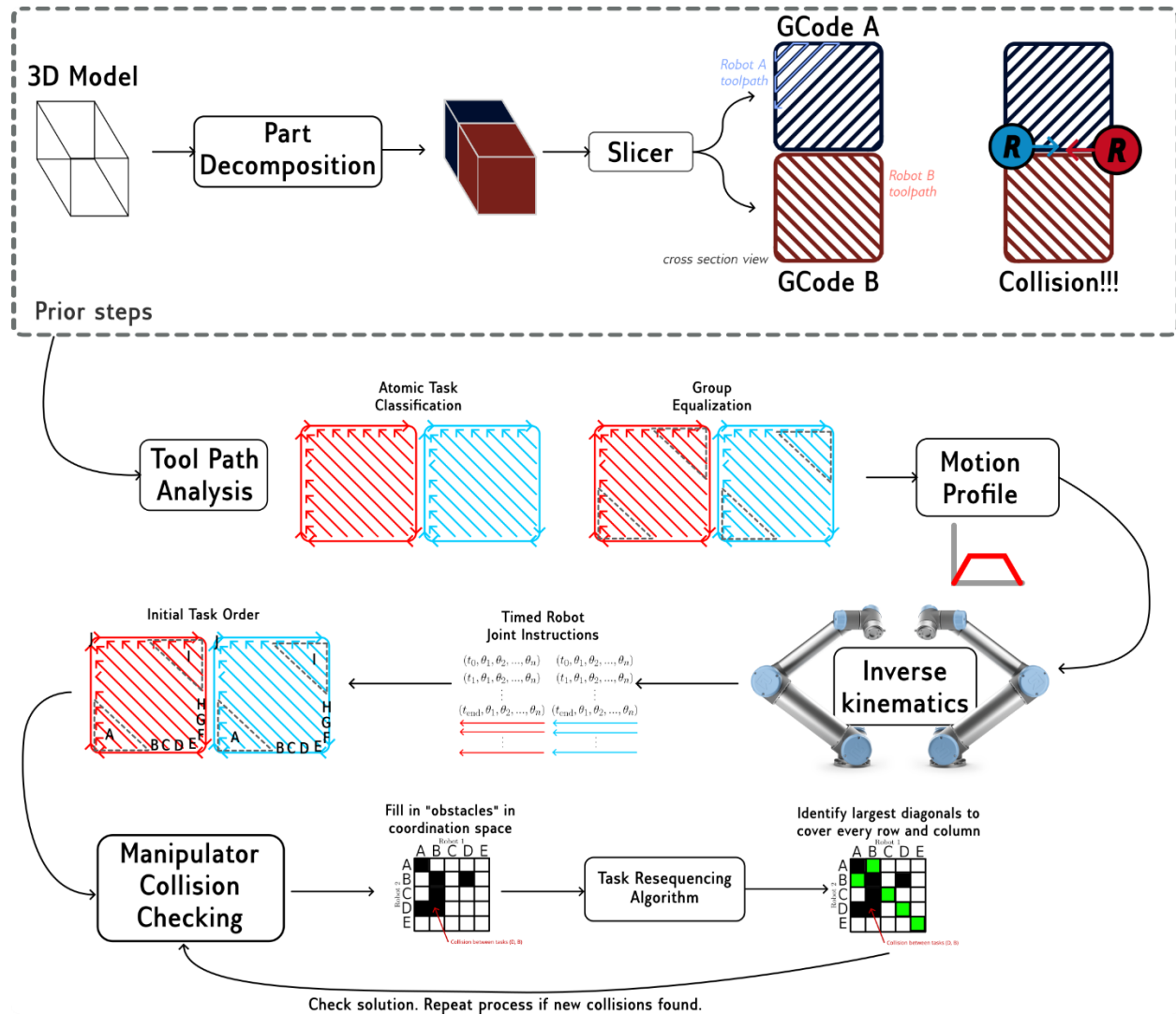


Figure 5: The complete pipeline for manufacturing a part, starting with part decomposition, then slicing, and robot coordination

Slicing Approach

There is already open-source slicing software available that has many years of support behind it. To recreate all the features that have been developed over the past many years would be outside the scope of a senior design project. As a result, one of these options, Cura, will be used as the basis to develop the more fully featured slicer. Cura consists of a separate front-end and back-end that communicate with sockets. The front-end is largely written in Python while the back-end is mostly written in C++. Most of the work to develop the multiplanar and nonplanar functionality will occur in the front-end. This will include performing the operations to modify the model prior to sending it to CuraEngine, applying the inverse of

the transformation to the G-code received from CuraEngine, and exposing configurability to the user.

Models used for slicing and 3D printing are typically stored in mesh files. One common type of mesh file is an STL, which consists of a list of vertices that are connected to form each face of the model. When these meshes are loaded into Cura, they are stored as an array of vertices, so to transform the model, the transformation must be applied to each vertex. Since the transformations that will be applied are inherently nonlinear, the line segments connecting each vertex will be approximations of the transformation. The resulting model is a collection of secant lines approximating a nonlinear function. As each secant line becomes infinitesimally small, the approximation becomes more accurate. While having infinitesimally small secant lines is not practical, STL files are designed to maximize the distance between vertices, which will not be sufficient for accurate transformations. As a result, additional vertices will need to be interpolated between the existing vertices in a mesh to create a sufficiently accurate approximation of the transformation.

The transformations themselves will be represented as a set of functions applied to each component of the vertex. Although most surfaces used to slice the models may be defined in the z-axis as a function of the x and y components, there should be a method to define the transformation in any axis, as this will allow for more customization. Having the ability to define a surface in the x-axis, y-axis, or some combination of all three will be important for defining arbitrary planes to slice along and will simplify the rotation process. Additionally, the gain of the transformation should be able to change with respect to the current position along one of the three axes. This is important for models that may have planar geometry at one end and nonplanar geometry as the model progresses. A square with a hemisphere on top would benefit from the ability to perform planar slicing for the square but follow a curved surface to print the hemisphere. Designing an appropriate gain function would allow for control over this. The inclusion of a modifiable gain also requires that the transformation be applied as a delta to the original point, rather than creating a new point entirely. The transformation equations incorporating these requirements are shown below.

$$Transformed_x = Original_x + Gain_x(x) * (f_x(x) + f_x(y) + f_x(z)) \quad (1)$$

$$Transformed_y = Original_y + Gain_y(y) * (f_y(x) + f_y(y) + f_y(z)) \quad (2)$$

$$Transformed_z = Original_z + Gain_z(z) * (f_z(x) + f_z(y) + f_z(z)) \quad (3)$$

These transformations are applied to the meshes in the Cura front-end before the vertices are transmitted to the CuraEngine. This is for simplicity, as the data structure representing the mesh is easier for the user to modify via changing options in the GUI. CuraEngine then can create 2D polygons from the transformed mesh to generate a toolpath pattern for each layer. CuraEngine then generates G-code for the planar toolpath and transmits

this G-code to Cura. Similar to STLs, G-code files are generated to use the minimum amount of movement commands. As a result, the lines generated for each command are rather long and must be interpolated to ensure that applying the inverse transformation will result in a geometrically accurate original model.

Since the transformations applied are inherently nonlinear, an iterative approximation process similar to the Newton–Raphson method is used to determine the original position that should be used. Since a delta is being applied to the original position, rather than the entire operation being a transformation, we can use a starting point to apply the transformation and determine the accuracy of our starting point. Then, the point can be modified and the process completed until the starting point converges to the original position.

Beyond the transformation and inversion of the mesh and G-code, several other modifications were required to ensure a proper slicing surface was generating and the resulting G-code was appropriate. One such modification is to the extrusion value generated with each G-code line. The extrusion value tells the extruder how many millimeters of filament to push forward. Since the transformation changes the distance between the original points the G-code was moving across to the new ones, the extrusion amount should change accordingly. To accomplish this, the extrusion value was scaled by the ratio between the previous and new distances.

To support effective surface selection, ease of use functionality and validation were exposed to the user. The ease-of-use functionality included the ability to configure a surface using either coefficients for a polynomial description of the surface or a lambda function. The inclusion of the lambda function as a possible surface descriptor means that the nonlinear functions will not necessarily be differentiable. This is a partial motivation for the inversion process described above. Additionally, the model is moved to the origin of the build plate prior to applying the transformation. This ensures that the coordinate systems between the model and world that the transformation is defined in are the same. This transformation is inverted, except for the z-position which is recorded, to ensure that the backend processes the model in the proper location. The z-position is not retained because some transformations lower portions of the model. These lowered portions could then be below the build plate, which would result in the backend not slicing them. Instead, the z-position is modified so the bottom of the model is touching the build plate. This change is inverted during the G-code inversion process. A final support option for user surface selection was added to ensure that the surface chosen will result in a valid slice of the model. For this, the change in the maximum model curvature was calculated, and if this change was too great, then the user was notified that the model shape would not likely be retained due to the filament thickness.

Motion Planning Approach

Since the motion planning of multiple collaborating arms is still an active research problem, it was necessary to develop a set of tools in which ideas and assumptions could be tested quickly. To facilitate rapid prototyping and experimentation, a bare bones 2D representation of the problem was developed to evaluate different motion planning algorithms. This simulation was implemented in C++ using the Simple and Fast Multimedia Library (SFML). Towards the conclusion of the project, a second iteration of the simulation was in development to address the performance shortcomings as well as generalizing the data model to handle more complex scenarios, such as multiple manipulators. While developing this second iteration, we simultaneously began implementation of the final motion planning system using ROS.

After conducting an extensive literature review, the two most promising motion planning methods were Robot Task Sequencing and Path Coordination. Space partitioning was rejected due to its difficulty as well as lack of generalization. Space partitioning restricts the motion of each robot to operate in a designated region. While it is the easiest algorithm to understand conceptually, the algorithms for processing three-dimensional geometry are extremely complex and would have required tight integration with the collision checking methods. The scale of this problem fell well outside the scope of what was feasible during this project. Additionally, the restrictive nature of space partitioning eliminates a large number of feasible and optimal solutions. Even if a robot arm crosses a partition, it does not mean that a collision is occurring. By considering a more abstract version of the problem which does not rely on geometric algorithms, we can solve a more general problem and avoid painful implementation details.

The first approach that was seriously considered was Robot Task Sequencing. This is a variation of the Traveling Salesman Problem (TSP), which is an NP-Hard problem. A solution to the TSP is a minimum-cost tour through a set of nodes on a graph. These nodes to visit are the set of minimal tasks that complete an entire 3D print. The toolpath plan from the slicer is analyzed and broken up into atomic segments that can be executed in any order without dramatically changing the manufactured part. By changing the order of the tasks, we can avoid collisions.

Some of the main challenges of coordinating the motion of multiple arms can be seen if we attempt to manufacture the cone benchmark in Figure 6. This is a cone broken up into 3 segments with each segment manufactured by a different arm. As the arms manufacture the part, the surface area of the layer decreases, and the potential for conflict increases. If the task order is poorly chosen a layer, the arms will enter a deadlock as they all slowly converge towards the center. This demonstrates that even if we inserted delays to avoid a collision, it

does not prevent an impending deadlock. A better way would be to determine the order in which to execute tasks so that we eliminate the deadlock and entirely remove the need for delays that slow down the manufacturing time.

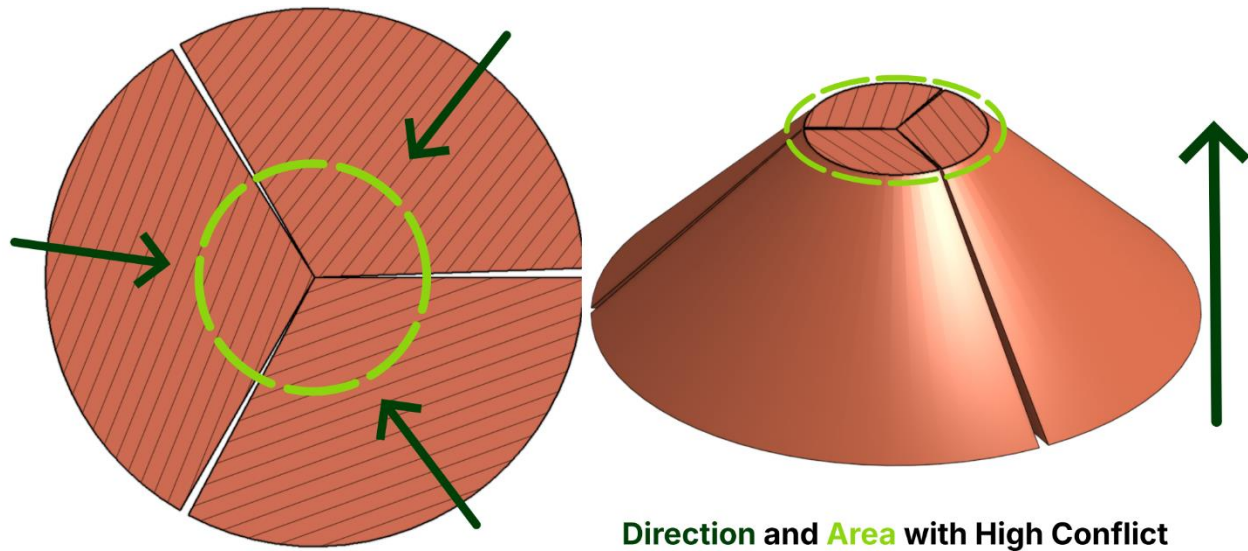


Figure 6: A 3D model of a cone that is segmented into three segments. As the robots manufacture, they converge towards the center into a deadlock. As the layer height increases, the layer area decreases, and conflict increases.

While changing the order of tasks can address the issue of deadlocking in this scenario, the computational load is high. Solvers that find optimal solutions to the TSP scale poorly as the number of tasks increases since the problem is combinatorial [3]. For any given layer there may be anywhere between 30 to hundreds of tasks. And a sliced 3D model may have hundreds of layers. Many task sequencing approaches would struggle to find a solution for a single layer in a reasonable amount of time, let alone a whole part. It is critical we limit processing time to not exceed several minutes since we risk taking more time to compute a plan than to manufacture the parts individually.

Task sequencing is not a catch all solution. While it may be able to find an ordering of tasks that minimizes deadlocks, the method can backfire and end up introducing many costly task switches in an effort to avoid collisions. In the initial tool path plan, the robot executes all the tasks sequentially. This has a minimal task switching cost since the initial ordering has the tasks close in proximity. Resequencing may require transitioning between two tasks that are on opposite sides of the workspace. This is why simply randomizing the order of tasks is extremely costly since most of the manufacturing time will be spent switching tasks.

While delays are not desirable, they are a preferable alternative to having an overload of task switches. The most promising delay-based method treats the problem as a motion planning problem in coordination space [4]. This method is highly general, avoids several classes of deadlocks, and is scalable to tens of robots. The general understanding of inserting

delays to avoid collisions is an intuitive solution, but the proper implementation that inserts delays in a smart and non-greedy manner is non-trivial.

Both coordination methods have their benefits and drawbacks. It is believed that if a combination of these two methods can be successfully integrated, it will serve as a robust solution to the robot arm coordination problem. Since this is a research-intensive aspect of the project with many unknowns, it was decided to first focus on the task resequencing approach as a proof of concept since it has a more straightforward implementation. In the future, the delay-based algorithm will be evaluated and integrated with the task sequencing.

Task Sequencing Problem Formulation

The key insight in our task sequencing formulation is that we do not need to use a traveling salesman solver. The tool path plan already contains the optimal sequence in which to execute tasks. The question becomes how to reduce the quality of this ordering in order to avoid collisions between other robots.

Our task sequencing solution borrows the idea of coordination space from the delay-based algorithm. Instead of using a continuous time representation, we can treat a task as the smallest unit of time. In Figure 7 we align the tasks of Robot 1 (A, B, C, D, E) with the tasks of Robot 2 (A, B, C, D, E) into a grid. It is important to clarify that Robot 1 task A is not the same as Robot 2 task A. The letter notation is simply used to represent the initial ordering of the tasks for robot.

A black colored cell in this space represents a conflict between a task pair. This could be a collision or some other constraint that we impose. An empty cell represents a valid pair of tasks without conflict.

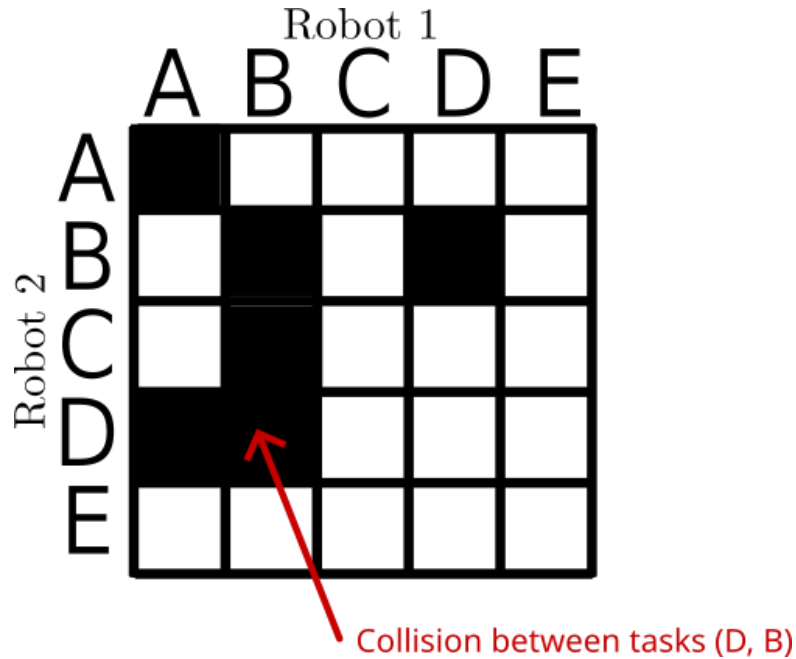


Figure 7: Coordination space representation of robot tasks. Robot 1 tasks are the columns and Robot 2 tasks are the rows. Black squares are collision between task pairs.

We assume that the robots cannot proceed onto the next task until all robots complete their current task. If this assumption was not in place, there would be ambiguity in how the task pairs align with one another. If there were a series of short tasks, Robot 1 might complete A, B, C in the time Robot 2 completes A. And if you reorder the tasks, this correspondence may change again. At least this way we maintain a one-to-one pairing of tasks that is independent of how we order the tasks.

This simplification causes one robot to pause until the other robot completes their task. To minimize this effect, we can use a knapsack algorithm to group together series of small tasks into a combined task.

Assuming there are no conflicts between two robots, we can visualize the optimal task ordering as a diagonal going from top left to bottom right. Or alternatively from bottom left to top right. Diagonals indicate optimality because we maintain as much of the original ordering as possible from the tool path plan (Figure 8).

It is important to note that a complete solution covers every row and column at least once. This corresponds to making sure that every task for every robot is executed at least once. If you fill in multiple cells within the same row or column, this will act as a delay that is the length of a task. Delaying for the entire duration of a task is extremely costly and is something to be avoided if possible.

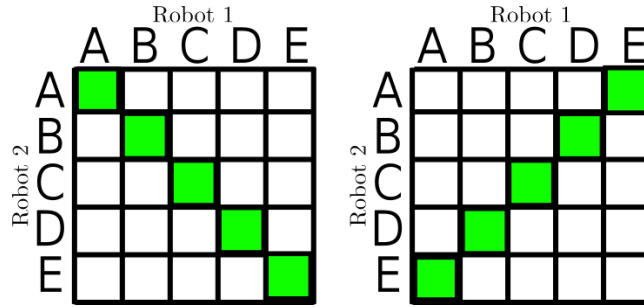


Figure 8: The optimal solution in a coordination space with no conflicts are the two main diagonals.

A transition between tasks in this formulation is viewed as a gap between two consecutive diagonals (Figure 9). These gaps need to be introduced to avoid regions of conflict in this space. We assume that transitions between tasks do not have any conflicts. While this is not necessarily true, especially for very large transitions, this can be practically achieved by applying an alternative motion planning algorithm, like an Artificial Potential Field (APF), to have the robots avoid each other while transitioning to their next task.

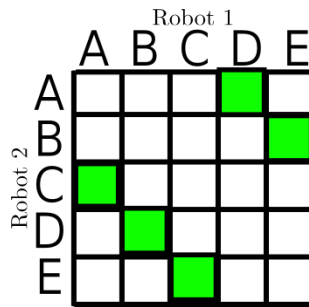


Figure 9: Gaps between diagonals represent a transition between tasks in coordination space.

There are several reasons why standard motion planning algorithms are undesirable to solve this problem. A motion planning algorithm may traverse around conflicts but introduce “repeated work” by filling in the same row or column multiple times, as mentioned before. Additionally, in coordination space we are searching for the all-pairs shortest path. We don’t necessarily have a fixed starting and ending pair of tasks that we want to reach. The main objective is to make sure that any sequence of tasks exists. And finally, encoding transitions between tasks scales extremely poorly because this ends up generating extremely densely connected graphs in for a pathfinding algorithm to use. This is because a solution can jump from diagonal to diagonal. To encode these transitions, you would need to connect the one cell to all other cells in the next row (or column depending on how you iterate). Assuming each robot has only N tasks, the number of edges for this N -by- N grid would be N^3 .

This problem of finding the optimal pairing between tasks can be understood as finding the maximum cardinality matching in a bipartite graph. This bipartite graph can be generated from our coordination space matrix by treating it as an adjacency matrix representation of a graph. Cells in which there is no conflict are represented as an edge between a task node for Robot 1 and the task node for Robot 2. Interestingly, as the complexity of the coordination space increases through more conflicts, the number of edges in the graph representation decreases.

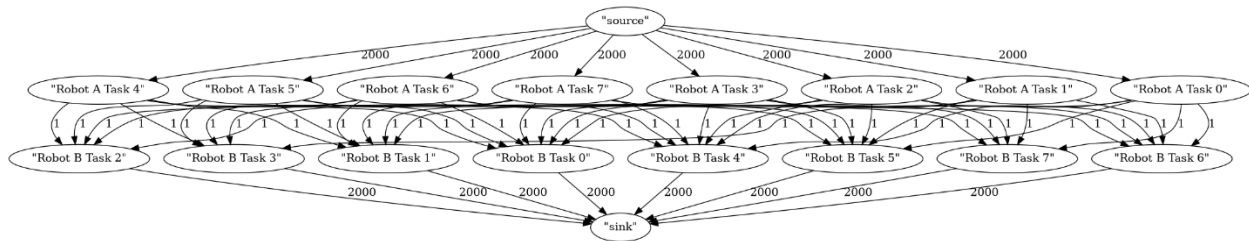


Figure 10: Coordination space representation converted into a bipartite graph matching problem to be solved using maximum flow.

Bipartite matching can be formulated as a max flow problem, for which there are extremely efficient algorithms that provide optimal solutions. As of 2022 there was a discovery that surprised the computer science community because of an almost linear time solution [5]. This is in contrast with the original algorithms like Ford-Fulkerson which are much slower. Unfortunately, even with fast max-flow algorithms, for $N > 2$ robots our coordination problem generalizes to finding a maximum cardinality matching in hypergraphs which is NP-Hard [6]. One potential way to find a sub-optimal solution is to solve the problem for expanding groups of two. If you have three robots, you could first solve for coordination between Robot 1 and Robot 2. Afterwards you treat Robot 1 and Robot 2 as a combined group and then solve for coordination between the group and Robot 3.

Project Management

Tasks and Subtasks

This senior design project is in collaboration with a CSE senior design team. The electrical engineering team will be responsible for the motion planning tools and algorithms and the slicing tools and algorithms. Meanwhile the CSE team will be responsible for the digital twin aspect of our project. The digital twin platform will be a set of tools allowing an operator to observe all aspects of the physical RAM system in a live simulation.

The motion planning tools and algorithms consist of creating a platform to rapidly test and develop motion planning algorithms. This has been executed through the creation of a 2D simulation, and it is essential for comparing different motion planning algorithms. Without this platform, the development time to implement each algorithm before knowing if it is effective would be substantially increased. Multiple motion planning algorithms also need to be evaluated to compare the trade-offs between planning efficiency, robustness, and ease of implementation. Once a motion planning algorithm is selected as the best option for the CRAM system, it needs to be implemented in the ROS environment being used for robot control.

The slicing tools and algorithms consist of developing a slicer that can support nonplanar and multiplanar slicing. An existing planar slicer will be chosen and have nonplanar and multiplanar functionality added. This allows the team to take advantage of the existing development work supporting slicing for 3D printers. The nonplanar and multiplanar slicing options will have to be added as configurations in the front end of the slicer. A structure and algorithm for performing the transformations and inverting them after the slice is completed will need to be implemented within the existing framework of the slicer.

A component closely intertwined with the nonplanar and multiplanar slicing components is part decomposition. This is the process of dividing the part into multiple segments to be sliced in different manners or divided between different robot arms. The implementation of the part decomposition tool is being performed by the CSE team; however, it will also be tied into the same slicing framework as the nonplanar and multiplanar slicer.

The responsibilities between both teams were closely intertwined and had a significant overlap in the area of testing on the robotic system and making modifications based on hardware results. Tasks such as operating the arms, modifying code for synchronization between both arms and the end effectors, and debugging extrusion issues were evenly distributed across the teams.

RACI Chart

Tyler Nardi is primarily responsible for the slicing methodology and implementation while Kalin Kochnev is responsible for the motion planning algorithms. We remained up to date with the developments made by the CSE team for the digital twin platform and part decomposition, as well as assisted with the integration of our contributions. Our faculty advisor Dr. Farhad Imani was made aware of any significant milestones and deliverables while our primary contact was the advising graduate student, Sean Rescsanki. The deliverables for the senior design class were split evenly between Kalin and Tyler, but Tyler was the appointed recipient for department communications. These responsibilities are outlined in the RACI chart below.

Table 2: RACI Chart for ECE Team 2522

Research	Tyler N.	Kalin K.	Dr. Imani	Sean R.	CSE Team	Shan Zuo
CRAM Review	Responsible	Responsible	Informed	Consulted	Responsible	Informed
FFF Review	Responsible	Responsible	Informed	Consulted	Responsible	Informed
STL Decomposition	Informed	Informed	Informed	Informed	Informed	Informed
Slicing	Responsible	Informed	Informed	Informed	Informed	Informed
Motion Planning	Informed	Responsible	Informed	Informed	Informed	Informed
Digital Twin	Informed	Informed	Informed	Informed	Responsible	Informed
Tasks						
Print Execution	Responsible	Responsible	Informed	Accountable	Responsible	Informed
Performance Evaluation	Responsible	Responsible	Informed	Consulted	Responsible	Informed
Scheduling	Responsible	Responsible	Informed	Accountable	Responsible	Informed
Department Communications	Responsible	Responsible	Informed	Informed	Informed	Informed
Website Upkeep	Responsible	Responsible	Informed	Informed	Informed	Informed
ECE Department Deliverables	Responsible	Responsible	Informed	Consulted	Informed	Accountable
ISCL Deliverables	Responsible	Responsible	Accountable	Consulted	Responsible	Informed
Second Arm Integration	Responsible	Responsible	Consulted	Consulted	Responsible	Informed
Slicer Development	Responsible	Informed	Informed	Consulted	Informed	Informed
Motion Planner Development	Informed	Responsible	Informed	Consulted	Informed	Informed
Joint Toolpath Planner	Responsible	Responsible	Informed	Consulted	Responsible	Informed
Decomposition Algorithm	Informed	Informed	Informed	Consulted	Responsible	Informed
Digital Twin Integration	Responsible	Responsible	Consulted	Consulted	Responsible	Informed

Timeline

There were several key milestones laid out for the fall and spring semesters to facilitate achieving our goals for the CRAM system. The goals for the fall were to have the slicing software in a functional state and to have a preliminary motion planning algorithm established. For the slicing system, the goal was to develop the foundational components that will be needed for nonplanar and multiplanar slicing. This includes the parameterization of the model, ensuring that the backend functionality is being properly utilized, and the inversion of the transformation process. Additionally, the CSE team worked to develop the decomposition tool that will be needed for multi-arm printing. For motion planning, the goals were to perform an extensive literature review of motion planning algorithms and assess their usefulness in our

case. The goal was to then begin the development of a simulation and visualization tool that could implement a greedy algorithm.

For the spring semester, slicing support was integrated into the front end for user access, as well as multiple features to increase the usability of the software. This included warnings for surface selection choices, support for rigid transformations, and adjustments to the extrusion amount. The spring semester also consisted of testing for nonplanar prints and fixing bugs that arose during the testing process. The motion planning algorithm was reworked during the spring semester as issues with the original strategy were found. This prompted an additional literature review that led to a new framework being developed to solve the problem. The rest of the semester consisted of working towards the implementation of this algorithm. Alongside this work, integration between the robot arms, hot ends, and digital twin components developed by the CSE team was performed. This work is documented in the GANTT chart below.

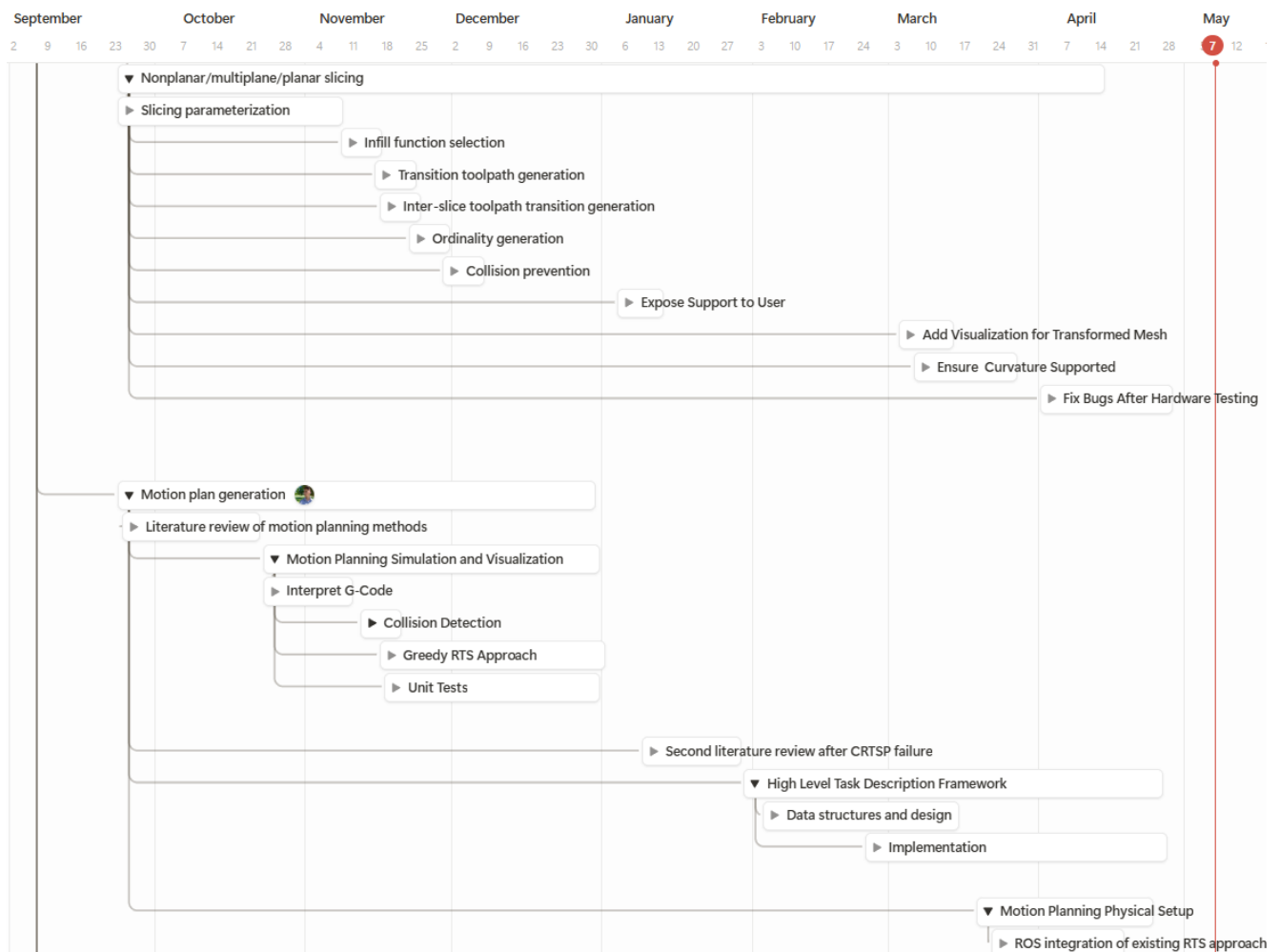


Figure 11: Gantt Chart for ECE Team 2522

Results

Slicing Results

The nonplanar slicing approach was successfully implemented with a variety of surfaces, including polynomial functions, trigonometric functions, and more complex lambda functions, such as the one used in the figure below. For this slicing operation, a lambda function was used to describe a triangular wave in the x-axis and y-axis. This surface was then used as the slicing surface for transforming the original model. It is important to note that this slicing surface is not differentiable; however, the model was still returned to its original structure due to the described inversion process.

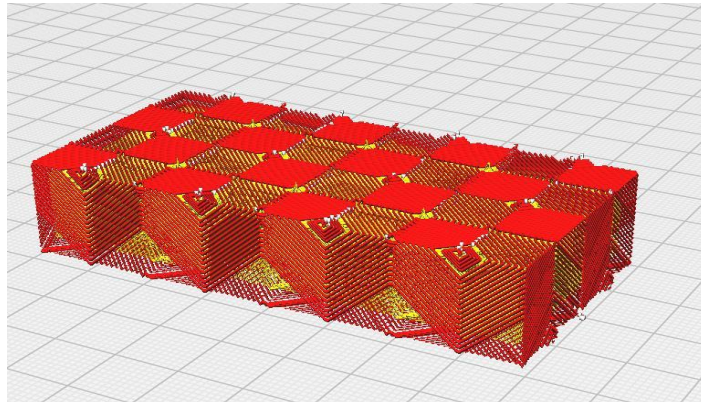


Figure 12: Rectangular Prism sliced using a lambda function representing a triangular waveform. The triangular waveform is propagated in the x-axis and y-axis. The lambda function is not differentiable, but the inversion process is successful in restoring the original model.

The slicing process has been tested with more complex structures that would benefit from a nonplanar manufacturing process. As shown in the figure below, an arch can be sliced using a quadratic surface to mirror the slope of the bottom surface. This allows for complex surfaces to be better constructed via FFF processes.

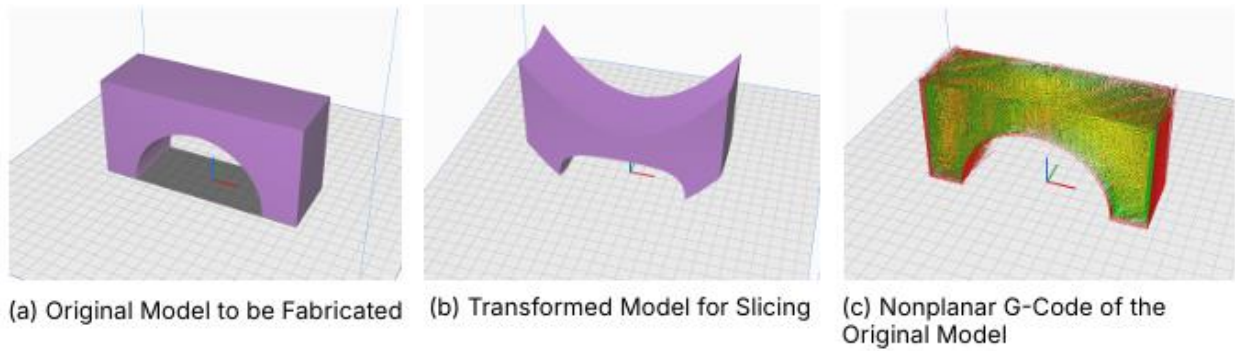


Figure 13: Slicing Process Shown for an arch being sliced with a quadratic function. (a) shows the original arch, and (b) shows the arch after being transformed before being sent to the backend. Finally, (c) shows the final g-code generated after the inversion is complete.

The different stages of the slicing process can be seen through Figure 13, with the first one being the deformation of the model following the defined surface. Once that stage is complete, the model is sent to CuraEngine, where the planar slicing occurs. The result is a set of G-code commands for manufacturing the deformed model with a planar toolpath. Finally, part (c) shows the nonplanar G-code that is generated for the original model through the inversion process.

The generated G-code files have been used to manufacture some models. Example fabricated parts can be seen in the figures below. As can be seen from Figure 15, modifying the extrusion factor is an important component. This model was manufactured without that modification implemented. The other model shown had the proper extrusion amounts, and these resulted in a higher quality print.

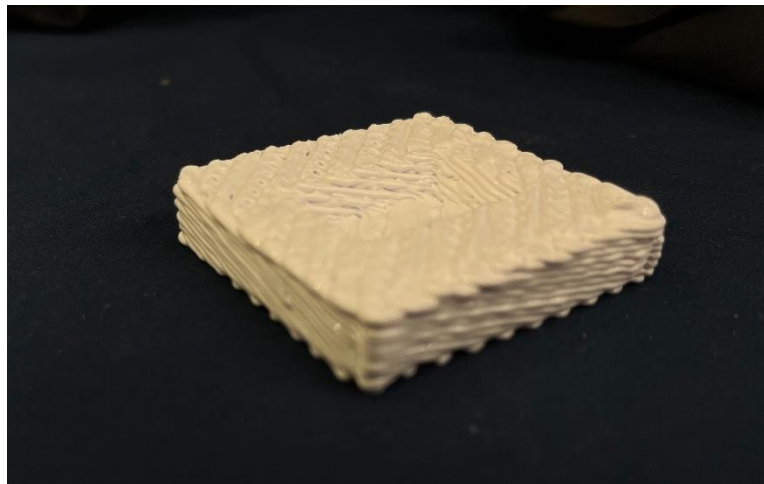


Figure 14: Nonplanar model manufactured with non-uniform surface.



Figure 15: Nonplanar manufacturing of a rectangular prism using a quadratic surface that required rotating the end effector.

An important inclusion is that for the fabrication of Figure 14, the hot end could be kept vertical without interfering with the printing process, as shown in Figure 16.



Figure 16: Extrusion of nonplanar model with the hot end vertical since the slicing surface used does not cause interference.

This is not the case for more complex slicing surfaces, such as for the model manufactured in Figure 17. For those, the nozzle was rotated based on the direction the arm is moving in. That direction is assumed to be the same as the direction the filament is extruded in because they are attached. Using this information, the nozzle is oriented to remain normal to this direction of motion. Since the motion is a vector, there is a plane of normal orientations. One is chosen such that it avoids interfering with the model location. The angle of the orientation is also limited to a 15° cone about the vertical z-axis to ensure that the arm does not excessively rotate and collide.



Figure 17: Manufacturing a nonplanar model with the extruder oriented normal to the motion and away from the existing model.

The fabrication of a model with this quadratic surface and other more complex surfaces is made possible through the reorientation of the hot end.

Motion Planning Results

The 2D simulator developed can view multiple robots executing G-code specified by a slicer and test a variety of task resequencing algorithms.

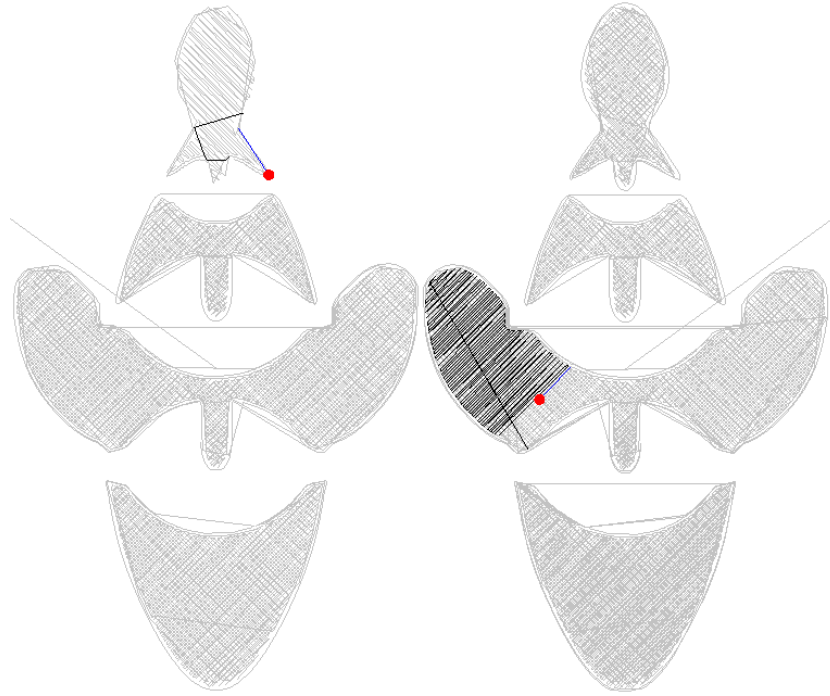


Figure 18: In red: robot, In black: path extruded by robot on current layer, In gray: previous layers extruded by robots.

The most important criteria to meet with the task resequencing algorithm is that it is complete (a solution can always be found if there is one) and it can scale several hundred tasks so it can be applied to hundreds of layers. To validate this, we generated randomized scenarios in coordination space. Two randomization methods were chosen. The algorithm was tested under uniform random noise where 30% of task pairs have a conflict. It was also tested under conditions where the conflicts are more regular. We generated rectangular regions whose width and height are randomly chosen but restricted to avoid exceeding a fixed length. Both the upper diagonal and lower diagonal solutions were generated.

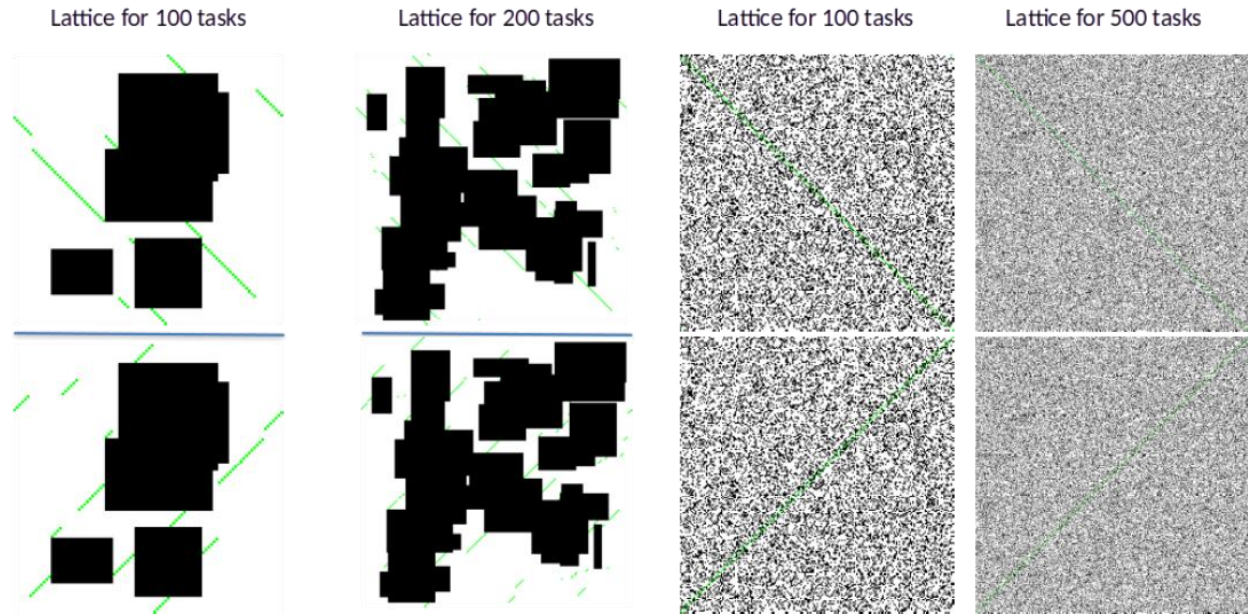


Figure 19: Upper and lower diagonal solutions for two different types of randomly generated coordination spaces. First two figure columns are randomly generated rectangular conflicts while next two figure columns are noisy coordination spaces.

As we can see the algorithm is capable of finding optimal solutions in very complex scenarios. It is also very scalable and finds the lower and upper diagonal solutions for 500 tasks in under a minute and 100 tasks in just a few seconds, even while using the less efficient Ford-Fulkerson algorithm. For our physical setup we most likely will never need to exceed more than 100 tasks. This demonstrates that this algorithm far surpasses our requirements.

The full implications of this stable matching problem in regard to solutions to the coordination problem is not described here and requires more study of the literature. The solutions to the matching problem form a lattice in which alternative solutions can be generated from [7]. The upper and lower diagonal solutions correspond to matching according to the preference of one robot over another. In other words, one robot will always end up getting the shorter end of the stick when it comes to solution quality. It is desirable to determine how to combine these solutions so that both robots have an agreeable outcome.

Another issue at hand is how to handle incomplete information or new information. In the above tests, we assume that the conflict regions are already known. In practice we must test for collisions between every pair of tasks. It may be more useful to test for collisions as needed rather than try to compute the conflicts ahead of time. If we assume the conflict regions represent collisions that are time-invariant, meaning at any point in time there is going to be a collision, then we don't need to change anything. However, because collisions are not just dependent on space but on position and time, it's possible that a conflict won't be found entirely because the robots weren't at the same place at the same time. So, by choosing not to

search the entire space, we only discover the conflicts that directly impede coordination, saving precious collision checking time.

And finally, we also hope to determine which task pairs impede the quality of the solution most. That way we can insert delays into that task pair using the delay-based coordination method.

Recent work appears to answer many of these questions regarding the stable matching problem [8]. The authors state they have not found a direct practical application of their work so this could lead to a future collaboration.

Summary

For our project, we were tasked with developing a CRAM system with a digital twin to support FFF manufacturing. This project was divided into 4 main components, part decomposition, slicing, motion planning, and the digital twin. The part decomposition and digital twin components were created by the CSE team, while the ECE team was responsible for the slicing and motion planning components. Part decomposition consists of segmenting a single model to slice in separate groups. This could be so that different arms can be responsible for certain components or so that different slicing methods could be used. Slicing is the process of generating the toolpath required to construct the part. To effectively utilize the robotic arms and increase part quality, nonplanar and multiplanar slicing techniques were used. The decomposition and slicing are incorporated into the existing Cura slicer. This provides a strong planar slicing framework that was used to more effectively develop these additional techniques. A simulation environment was developed to test task resequencing algorithms. A robust task resequencing algorithm was found that uses stable matching to have a guaranteed and optimal solution for two robots. The resequencing approach is still in the process of being ported to ROS. The digital twin was developed in ROS to simulate the print process with real-time data and control the arms. It currently has a component to track the end effector position as the part is manufactured, and integrated control for the hot end alongside the arm controls. Sensor data from the thermal camera was also analyzed to determine the best method for integrating into the digital twin.

Appendix

Senior Design Checklist

Project name: Digital Twin for Cooperative Robotic Additive Manufacturing

Sponsor: School of Mechanical, Aerospace, and Manufacturing Engineering

Team members (majors/programs): Tyler Nardi (Electrical Engineering, Computer Engineering),
Kalin Kochnev (Robotics Engineering)

Faculty advisor(s): Shan Zuo

Skills, Constraints, and Standards: *(Please check (v) all those that apply to your project.)*

Skills:	<i>Please check (v) all those that apply to your project</i>
Analog circuit design and troubleshooting	
Digital circuit design and troubleshooting	
Software development/programming	✓
Embedded Systems/Microcontrollers	✓
Web design	
RF/wireless hardware	
Control systems	✓
Communication systems	✓
Power systems	
Signal processing	
Machine shop/mechanical design	✓
Other (please specify):	
Constraints/factors:	<i>Please check (v) if applies and briefly explain</i>

Economic (budget)	
Health/safety	<i>✓ We had to ensure the materials being manufactured were non-toxic. This restricted us to filaments like PLA.</i>
Manufacturability	
Environmental (e.g., toxic materials, fossil fuels)	
Social/legal (e.g., privacy)	
Other (please specify):	
Standards:	<i>If applicable, list the name or # here:</i>
List all standards /electric codes that you used (e.g., IEEE 802.11, Bluetooth, RS-232, VHDL, etc.) or that are key specs of the components	N/A

Bibliography

- [1] S. Rescanski, R. Hebert, A. Haghighi, J. Tang, and F. Imani, "Towards Intelligent Cooperative Robotics in Additive Manufacturing: Past, Present and Future," Aug. 09, 2024, arXiv: arXiv:2408.04827. doi: [10.48550/arXiv.2408.04827](https://doi.org/10.48550/arXiv.2408.04827).
- [2] D. Li, H. Su, X. Xu, Q. Wang, J. Qin, and W. Zou, "Cooperative Task Scheduling and Planning Considering Resource Conflicts and Precedence Constraints," *International Journal of Precision Engineering and Manufacturing*, vol. 24, no. 9, pp. 1503–1516, Sep. 2023, doi: [10.1007/s12541-023-00790-y](https://doi.org/10.1007/s12541-023-00790-y).
- [3] S. Alatartsev, S. Stellmacher, and F. Ortmeier, "Robotic Task Sequencing Problem: A Survey," *J Intell Robot Syst*, vol. 80, no. 2, pp. 279–298, Nov. 2015, doi: [10.1007/s10846-015-0190-6](https://doi.org/10.1007/s10846-015-0190-6).
- [4] P. Lertkultanon, J. Yang, H. Pham, and Q.-C. Pham, "Departure and Conflict Management in Multi-Robot Path Coordination," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, Brisbane, QLD: IEEE, May 2018, pp. 4327–4333. doi: [10.1109/ICRA.2018.8460587](https://doi.org/10.1109/ICRA.2018.8460587).
- [5] L. Chen, R. Kyng, Y. P. Liu, R. Peng, M. P. Gutenberg, and S. Sachdeva, "Maximum Flow and Minimum-Cost Flow in Almost-Linear Time," Apr. 22, 2022, arXiv: arXiv:2203.00671. doi: [10.48550/arXiv.2203.00671](https://doi.org/10.48550/arXiv.2203.00671).
- [6] E. Großmann, F. Joos, H. Reinstädter, and C. Schulz, "Engineering Hypergraph β -Matching Algorithms," Aug. 13, 2024, arXiv: arXiv:2408.06924. doi: [10.48550/arXiv.2408.06924](https://doi.org/10.48550/arXiv.2408.06924).
- [7] C. Blair, "Every finite distributive lattice is a set of stable matchings," *Journal of Combinatorial Theory, Series A*, vol. 37, no. 3, pp. 353–356, Nov. 1984, doi: [10.1016/0097-3165\(84\)90056-6](https://doi.org/10.1016/0097-3165(84)90056-6).
- [8] R. R. Gangam, T. Mai, N. Raju, and V. V. Vazirani, "The Stable Matching Lattice under Changed Preferences, and Associated Algorithms," Jul. 20, 2023, arXiv: arXiv:2304.02590. doi: [10.48550/arXiv.2304.02590](https://doi.org/10.48550/arXiv.2304.02590).